

Memory

2026 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

Lecture 12

1.0.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Page Allocation Uses A Free List

Given physical pages, the operating system maintains a free list (linked list)

The unused pages themselves contain the next pointer in the free list

Physical memory gets initialized at boot

To allocate a page, you remove it from the free list

To deallocate a page you add it back to the free list

Back to Creating A Process

A large virtual address space is allocated "for free" to a process.

Page table? (1 page table per process).

RAM allocation? smallest unit = 1 page. content not populated.

↓
user req. memory. (4KiB).

(malloc) → page fault → request to kernel.
n × pages.

2³².
address space.

Back to Forking A Process

Copy-on-write

Page table? (+1, content copied).

RAM allocation? child shares RAM w/ parent.

why do we need to copy the page tables?

We don't! There's a system call for that — vfork

vfork shares all memory with the parent

It's undefined behavior to modify anything

Only used in very performance sensitive programs

Demand Paging

We use memory as a cache for the file system

Map memory pages to file system blocks

Only load them into memory when they're used through the page fault handler

If the page doesn't represent a file, we can map it to swap space

Move it to disk if we need to use more physical memory

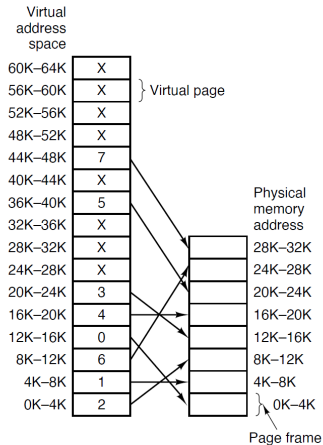
A Processes' Working Set Should Fit in Physical Memory

→ Given an amount of time, the number of pages your process uses is called its *working set*

If you cannot fit your working set into physical memory your process will thrash

It's constantly moving entries in and out of cache

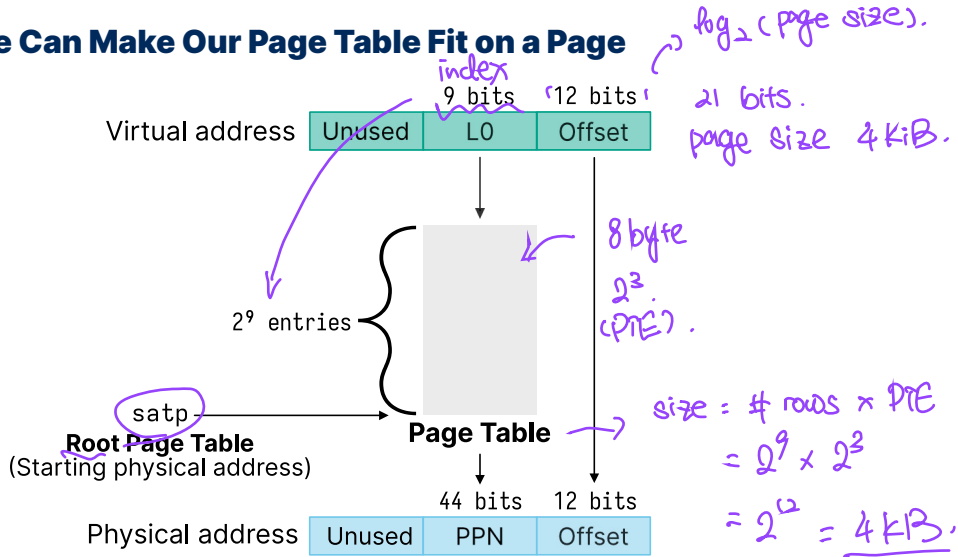
Not all virtual memory is mapped to physical memory



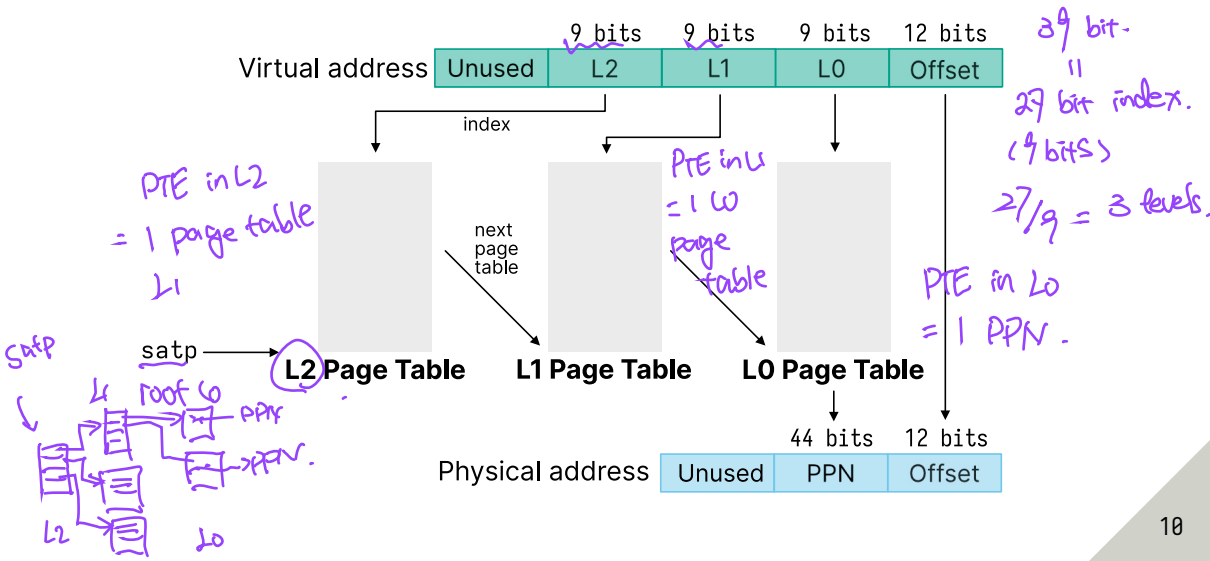
What Should We Do About the Page Table Size?

Most programs don't use all the virtual memory space, how can we take advantage?

We Can Make Our Page Table Fit on a Page



Multi-Level Page Tables Save Space for Sparse Allocations



Consider Just Two Levels

Let's consider a 30-bit virtual address with 4kB page.

Single-Level Page Table

of PTE.

$$2^{18}.$$

Size of PTE 2^3 .

Size of PT (fixed)

$$2^{18} \times 2^3 = 2^{21} \text{ Byte}$$

$$= \boxed{2 \text{ MB}}$$

offset : 12

index : $30 - 12 = 18 \text{ bits}$

$18 / 9 = 2 \text{ levels}$.

Multi-Level Page Table

minimum size :

2 pages . 8 KiB . 1 Li. (overhead) .

1 Lo

Maximum size :

2 MB + 4 kB .

Lo (data) , 1 Li (overhead) ,

Consider Just Two Levels

Assume our process uses just one virtual address at 0x3FFFF008
or 0b11_1111_1111_1111_1111_0000_0000_1000
or 0b11111111_11111111_00000000_1000) *offset*

We'll just consider a 30-bit virtual address with a page size of 4096 bytes
We would need a 2 MiB page table if we only had one ($2^{18} \times 2^3$)

Instead, we have a 4 KiB L1 page table ($2^9 \times 2^3$) and a 4 KiB L0 page table
Total of 8 KiB instead of 2 MiB

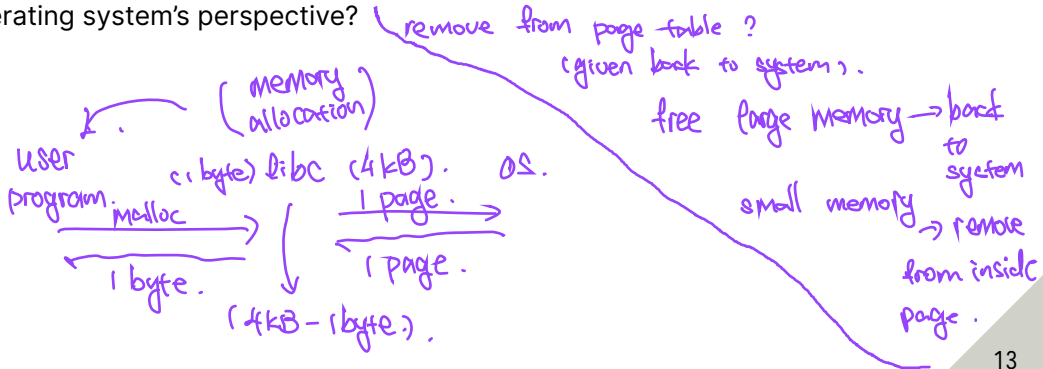
Note: worst case if we used all virtual addresses we would consume 2 MiB + 4 KiB

index
s1: satp → addr of table. s3: (0x1ff) L0
s2: index of L1.
0x1ff → addr. L0.
↓
PPN.

Back to malloc and free

What happens when you malloc memory?

=> If a process "shrinks" its heap at the application level (e.g., via free() calls), does that actually reduce the process's overall memory usage from the operating system's perspective?



Practice Question



$\log_2(5)$

2 bit 4

3 bit 8

0b111 $\rightarrow$ 8th entry in the table.

0b000 $\rightarrow$ 0b111. entry in the table.

Consider a system with 4096-byte pages, a PTE size of 6 bytes, 42-bit virtual addresses, and 48-bit physical addresses.

(a) How many offset bits are there in the virtual address?

$12 \rightarrow \log_2(\text{page size}) = \log_2(4096) = 12$

(b) How many bits of the virtual address are left over for index bits?

$42 - 12 = 30$

(c) For a single page table, how large would the table need to be?

$2^{30} \times \text{PTE} = 3 \times 2^{31}$

(d) How many PTEs could fit on a single page?

(e) How many levels of page tables would you need to ensure each page table fits on a single page?

multi-level

(d). $\frac{4096}{6}$ page size
size of PTE
 $= 682.667$
 $= 682$

(e). $\log_2(682) = 9$
9 bit 512
10 bit 1024

of levels:

$30 / 9 = 4 \text{ levels}$

Alignment: Memory Eventually Lines Up with Byte 0

If pages are *4096 byte aligned* in memory it means pages always start when the lower 12 bits are zero, in computing we like alignment

If a page started at address 0x7C00 its last byte would be at address 0x8BFF

Instead, a page would start at 0x7000 and end at 0x7FFF

Question: Is address 0xEC 8 byte aligned?

Alignment Example

```
struct AlignmentDemo
{
    char char_a;
    int  int_a;
    char char_b;
    char char_c;
    char char_d;
    int  int_b;
    int  int_c;
};
```

Using the Page Tables for Every Memory Access is Slow

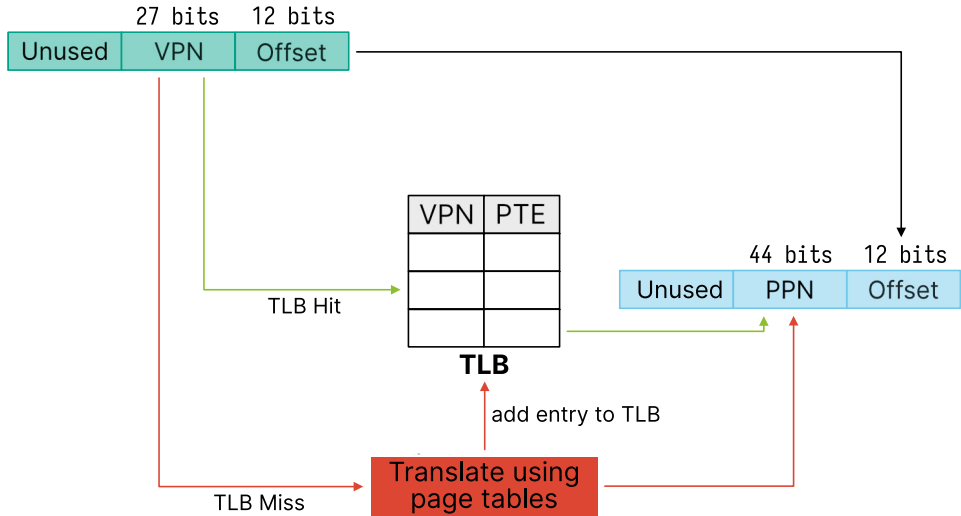
We need to follow pointers across multiple levels of page tables!

We'll likely access the same page multiple times
(close to the first access time)

A process may only need a few VPN $\rightarrow$ PPN mappings at a time

Our solution is another computer science classic: caching

A Translation Look-Aside Buffer (TLB) Caches PTEs



Effective Access Time (EAT)

Assume a single page table

(there's only one additional memory access in the page table)

$$\text{TLB_Hit_Time} = \text{TLB_Search} + \text{Mem}$$

$$\text{TLB_Miss_Time} = \text{TLB_Search} + 2 \times \text{Mem}$$

$$\text{EAT} = \alpha \times \text{TLB_Hit_Time} + (1 - \alpha) \times \text{TLB_Miss_Time}$$

If $\alpha = 0.8$, $\text{TLB_Search} = 10 \text{ ns}$, and accesses take 100 ns , calculate EAT

Context Switches Require Handling the TLB

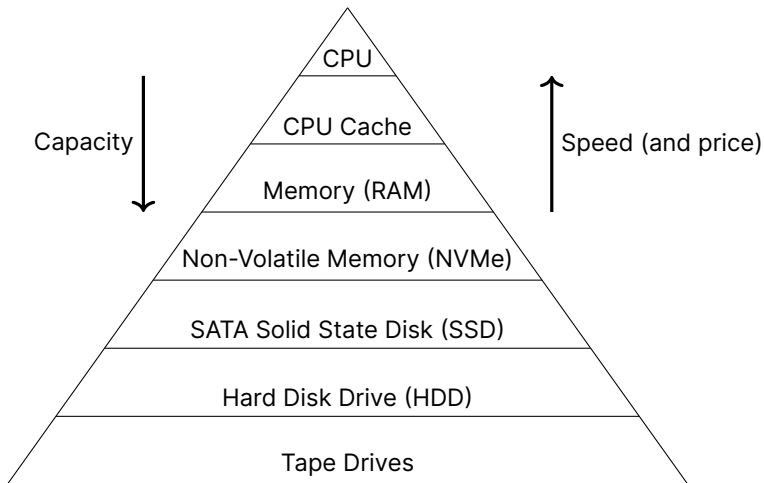
You can either flush the cache, or attach a process ID to the TLB

Most implementation just flush the TLB

RISC-V uses a `sfence.vma` instruction to flush the TLB

On x86 loading the base page table will also flush the TLB

Computer Memory Hierarchy is a Trade-off of Capacity and Speed



We Want to Hide the Hierarchy from the User

Each level wants to pretend it has the speed of the layer above it
and the capacity of the layer below it

The memory used by all the processes may exceed the amount of physical
memory

Not all of them may be in use at the same time

Only keep referenced pages in memory, put others on disk

Swap pages back to memory when they're needed

Page Replacement Algorithms

Optimal

Replace the page that won't be used for the longest

Random

Replace a random page

First-in First-out (FIFO)

Replace the oldest page first

Least Recently Used (LRU)

Replace the page that hasn't been used in the longest time

Page Replacement Evaluation

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

We'll use this for a bunch of examples during this lecture

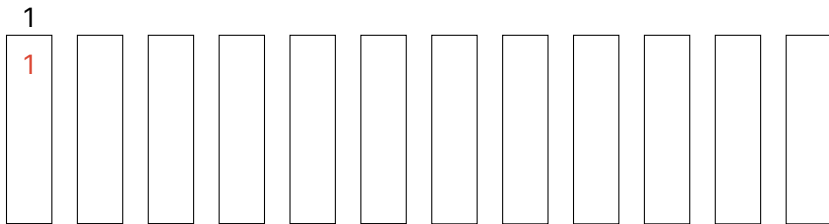
We want the fewest number of page faults

For every example we'll find the number of page faults

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

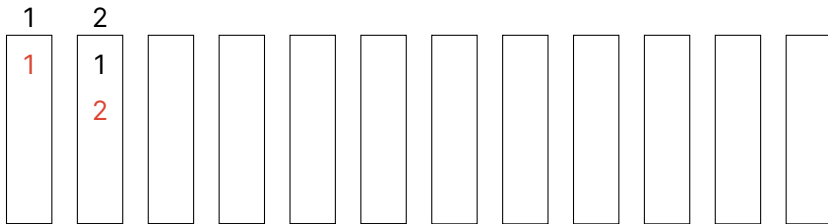
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

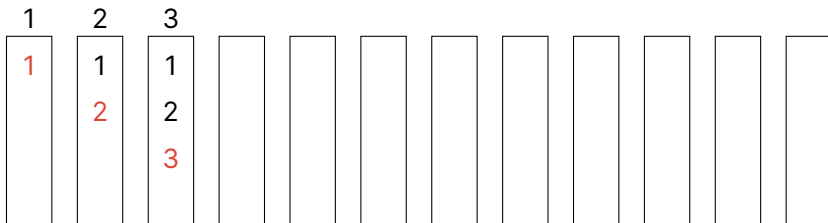
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

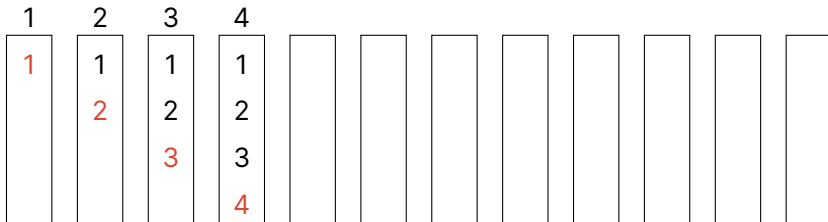
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

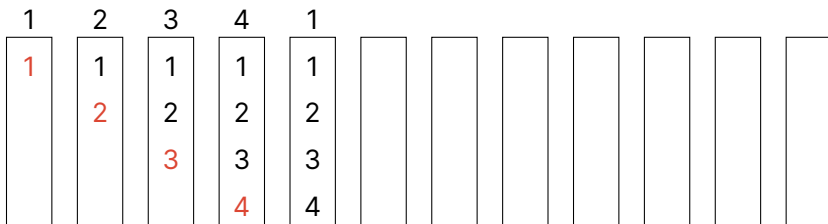
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

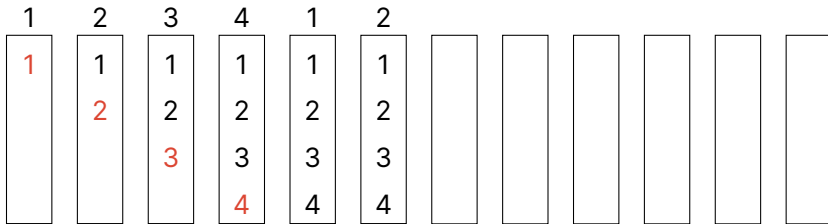
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

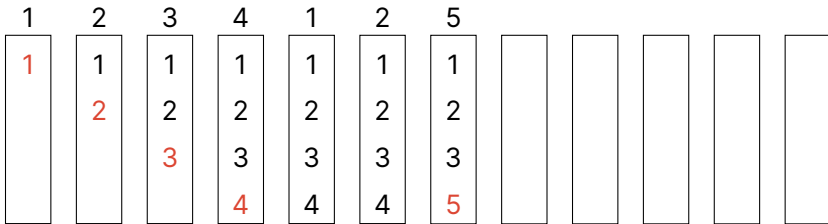
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

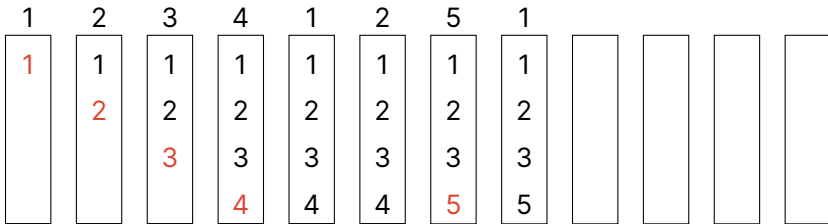
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

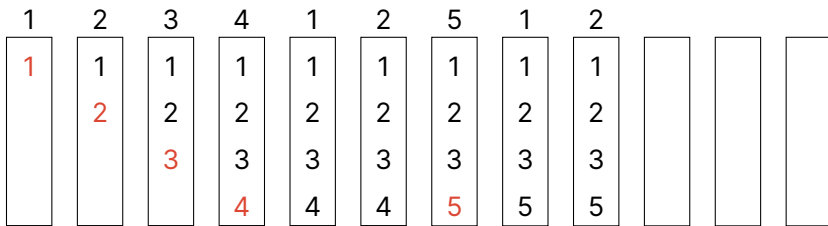
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3		
1	1	1	1	1	1	1	1	1	1		
	2	2	2	2	2	2	2	2	2		
		3	3	3	3	3	3	3	3		
			4	4	4	5	5	5	5		

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	1	1	1	1	4	
	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	3	3	3	3	3	
			4	4	4	5	5	5	5	5	

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	4	4
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	4	4	5	5	5	5	5	5

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

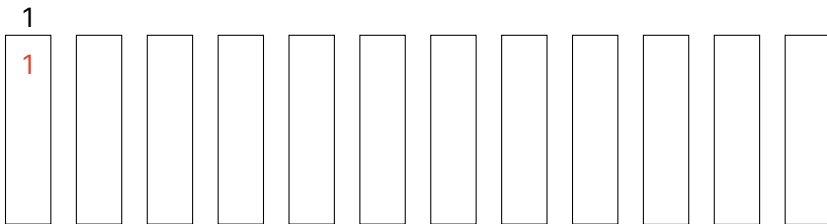
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	4	4
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	4	4	5	5	5	5	5	5

6 page faults

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

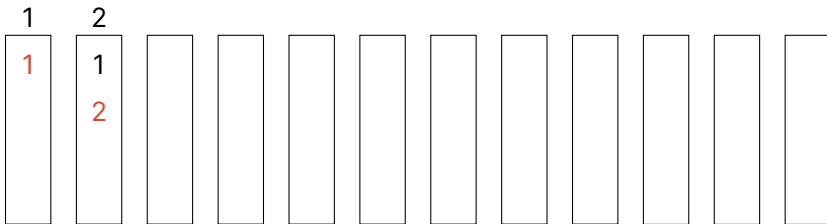
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

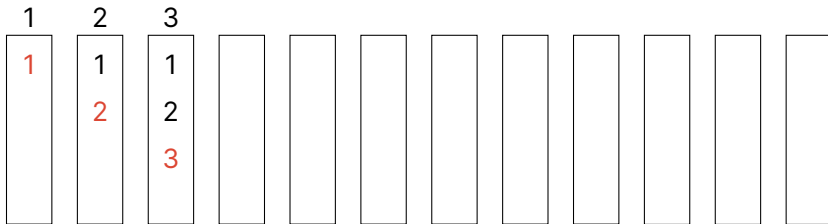
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

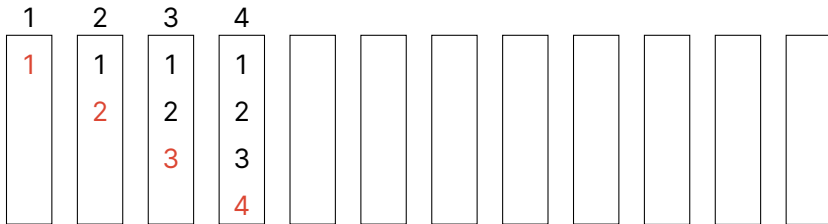
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

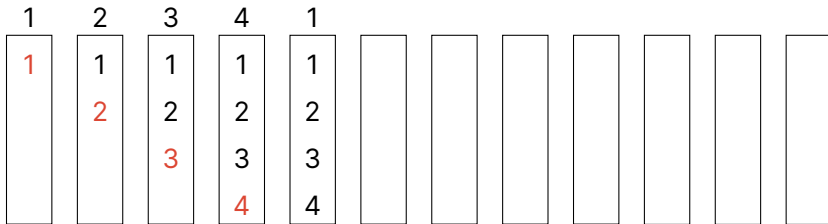
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

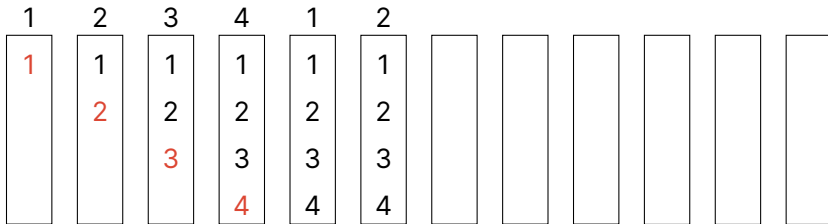
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

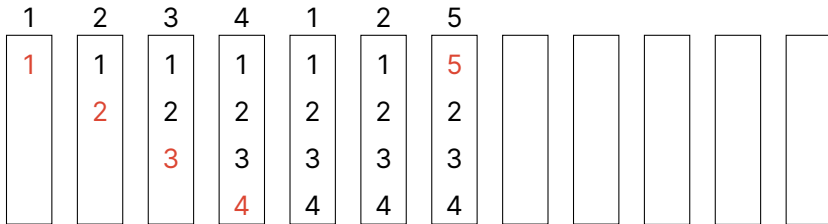
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

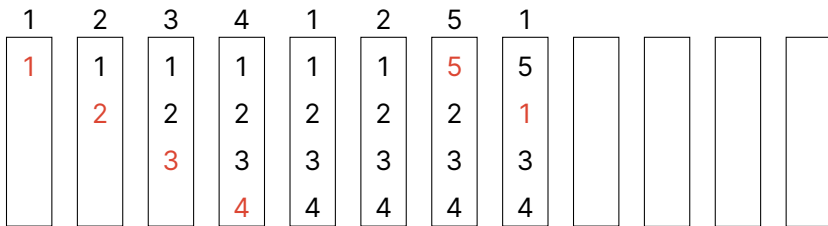
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

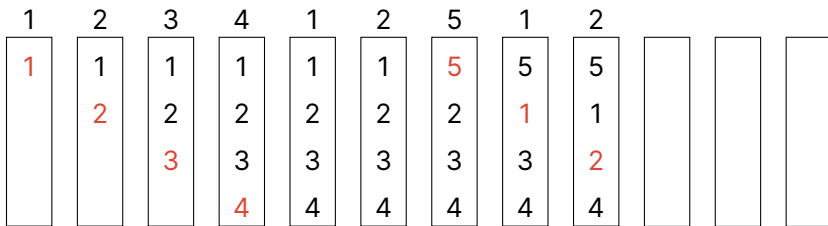
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

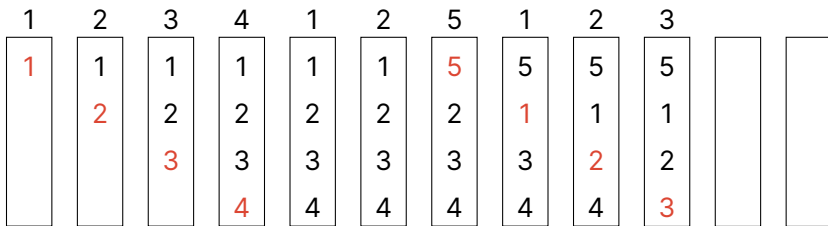
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	5	5	5	5	4	
	2	2	2	2	2	2	1	1	1	1	
		3	3	3	3	3	3	2	2	2	
			4	4	4	4	4	4	3	3	

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

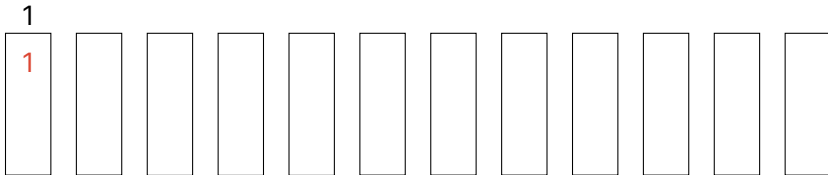
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

10 page faults

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

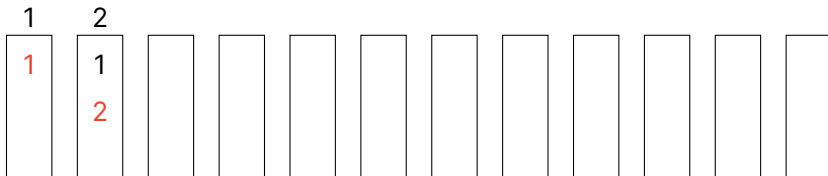
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

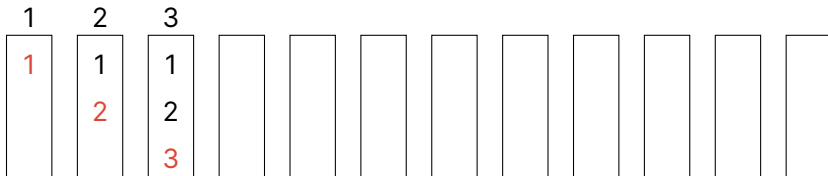
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

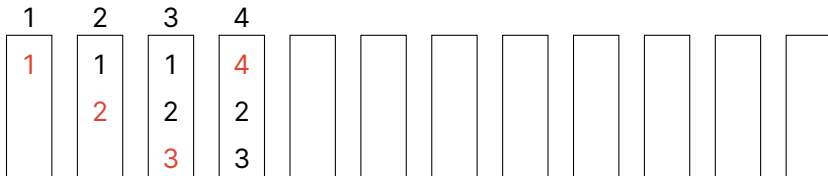
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

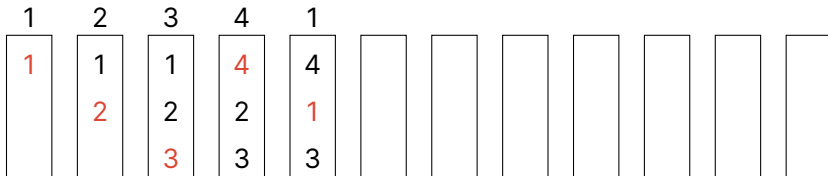
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

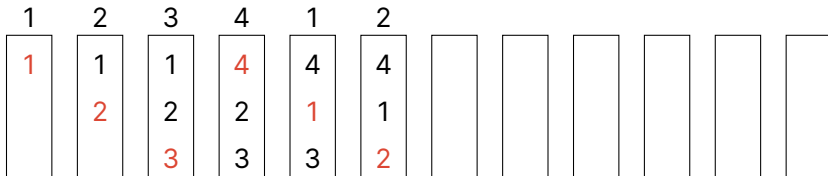
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

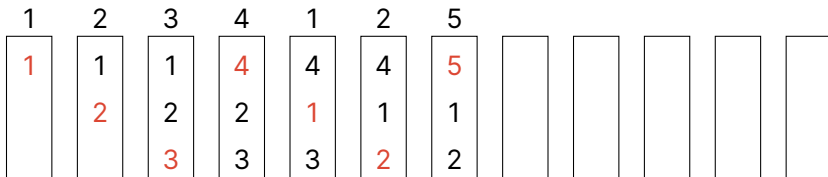
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

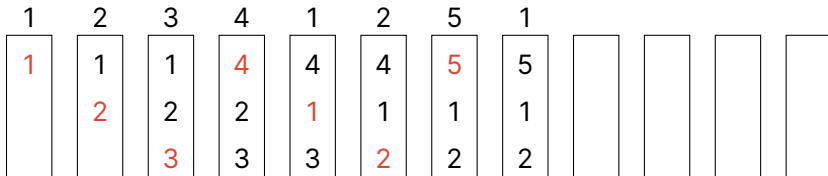
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

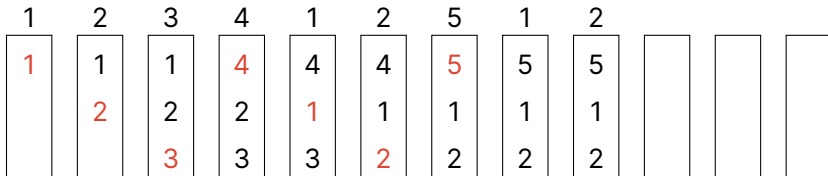
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3		
1	1	1	4	4	4	5	5	5	5		
	2	2	2	1	1	1	1	1	3		
		3	3	3	2	2	2	2	2		

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	4	4	4	5	5	5	5	5	
	2	2	2	1	1	1	1	1	3	3	
		3	3	3	2	2	2	2	2	4	

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4

9 page faults

Bélády's Anomaly Says More Page Frames Causes More Faults

This is a problem with FIFO algorithms

Does not exist with LRU or “stack-based algorithms”

Paper in 2010 found that this FIFO anomaly is unbounded

(<https://arxiv.org/abs/1003.1336>)

You could construct a sequence to get any arbitrary page fault ratio

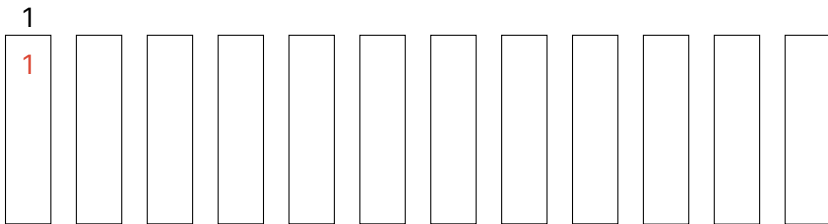
For other algorithms:

increasing the number of page frames decreases the number of page faults

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

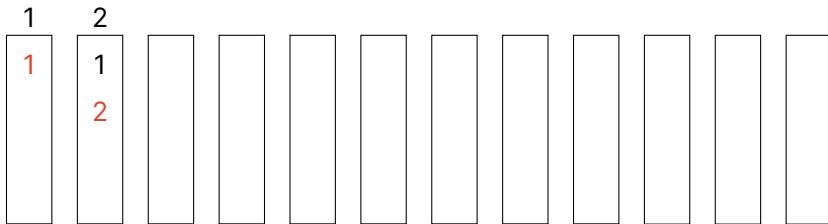
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

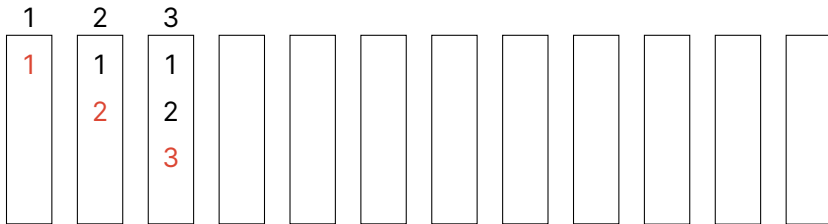
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

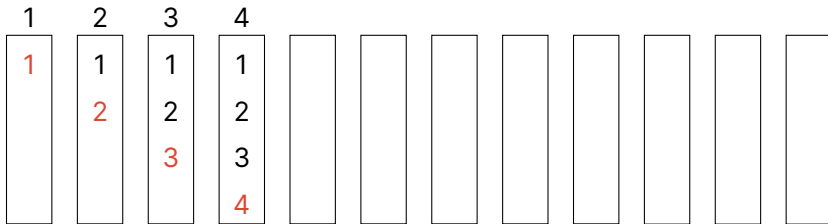
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

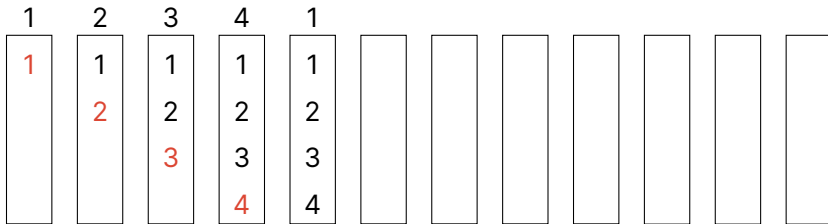
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

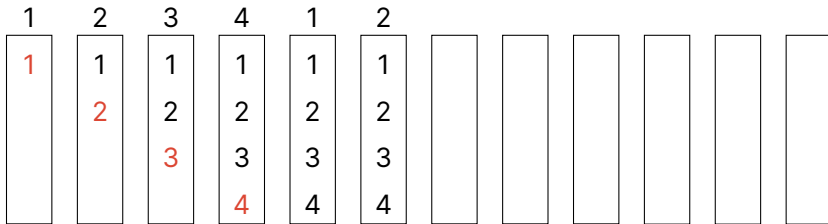
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

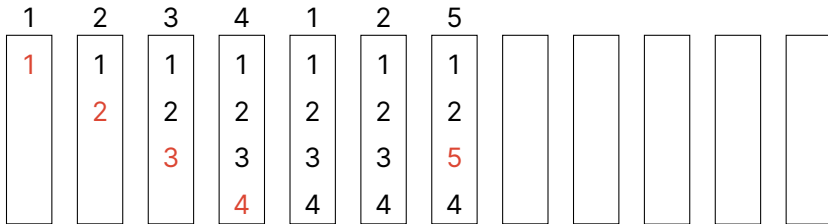
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

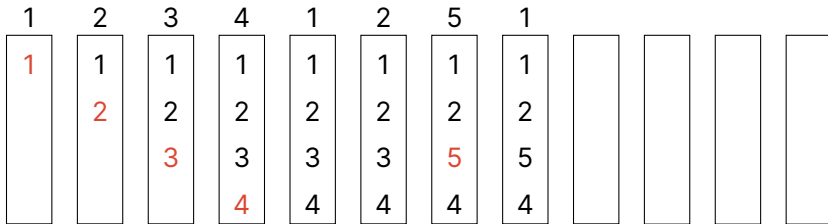
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

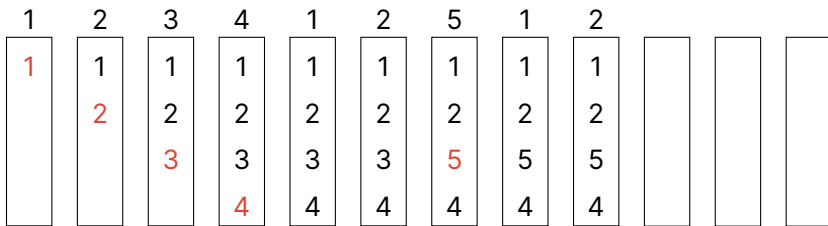
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

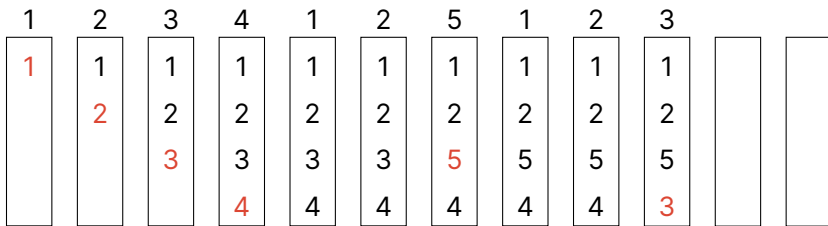
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	1	1	1	1	1	
	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	5	5	5	5	4	
			4	4	4	4	4	4	3	3	

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

8 page faults

Implementing LRU in Hardware Has to Search All Pages

You could implement it by keeping a counter for each page

For each page reference, save the system clock into the counter

For replacement, scan through the pages and find the one with the oldest clock

Implementing LRU in Software is Too Expensive

- Create a doubly linked list of pages

- For each page reference, move it to the front of the list

- For replacement, remove from the back of the list

- It requires 6 pointer updates for each page reference, and
also creates a high contention bottleneck for multiple processors

Implementing LRU in Practice Isn't Going to Work

We settle for approximate LRU

- LRU is an approximation of the optimal case anyways

There's lots of different tweaks you can do to implement it more efficiently

We'll be looking at the clock algorithm, but there's also:

- Least Frequently Used (LFU), 2Q, Adaptive Replacement Cache (ARC)

Implementing LRU in Practice Isn't Going to Work

We settle for approximate LRU

- LRU is an approximation of the optimal case anyways

There's lots of different tweaks you can do to implement it more efficiently

We'll be looking at the clock algorithm, but there's also:

- Least Frequently Used (LFU), 2Q, Adaptive Replacement Cache (ARC)

Clock Algorithm

Data structures:

- Keeps a circular list of pages in memory
- Uses a reference bit for each page in memory (light grey in next slides)
- Has a “hand” (iterator) pointing to the last element examined

Algorithm, to insert a new page:

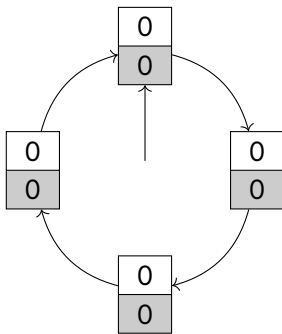
- Check the hand's reference bit, if it's 0 then place the page and advance hand
- If the reference bit is 1, set it to 0, advance the hand, and repeat

For page accesses, set the reference bit to 1

Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

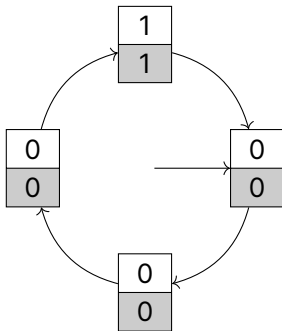


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1

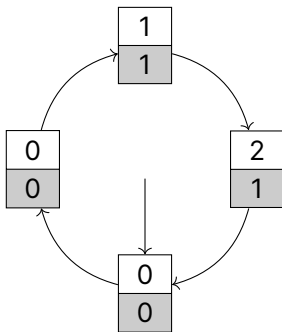


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2

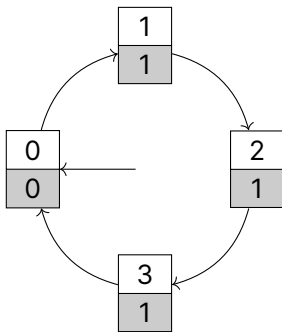


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3

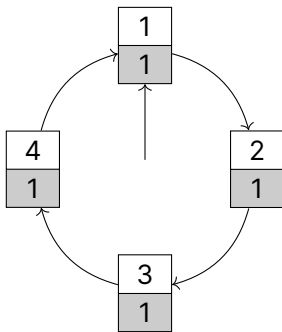


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4

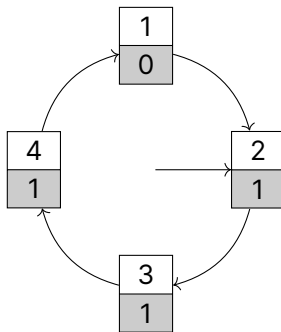


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

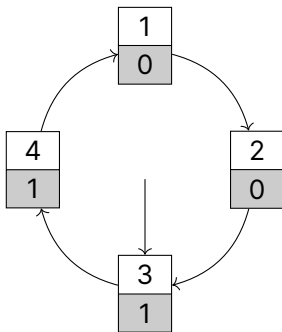


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

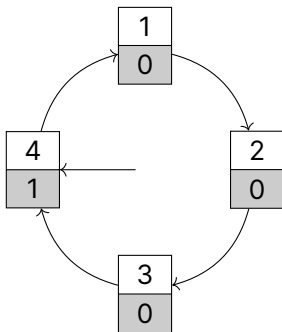


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

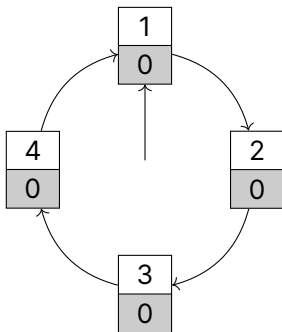


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

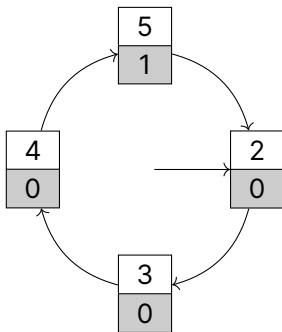


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

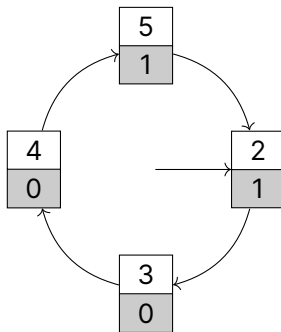


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2

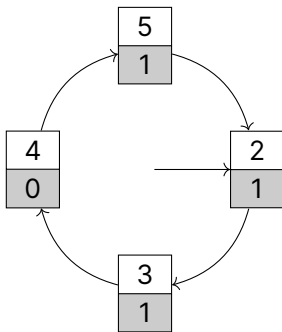


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3

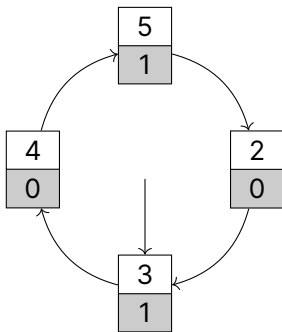


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

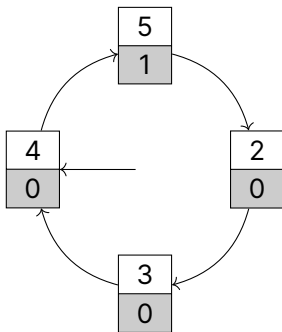


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

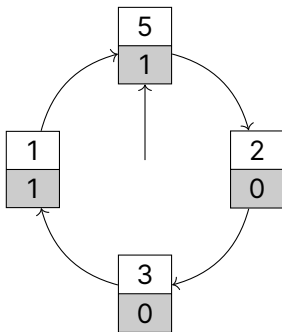


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

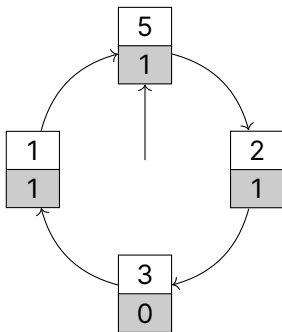


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1 2

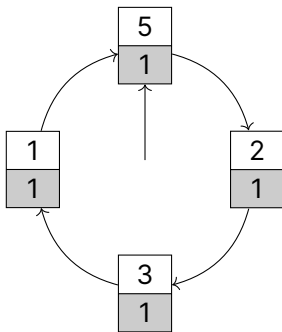


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1 2 3



Clock Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1	2	3	4	5	2	3	1	2	3
1	1	1	1	5	5	5	5	5	5
	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3
			4	4	4	4	1	1	1

6 page faults